

Quad Core Machine

Threads & Concurrency

Input Array



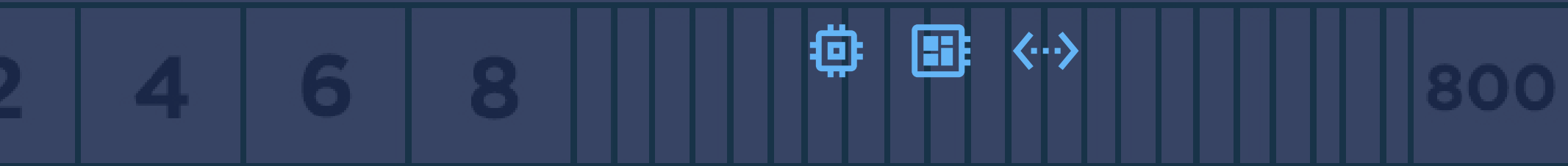
Output Array

4 sec



Mata Kuliah Sistem Operasi

without threads QuadCore Machine



Input Array

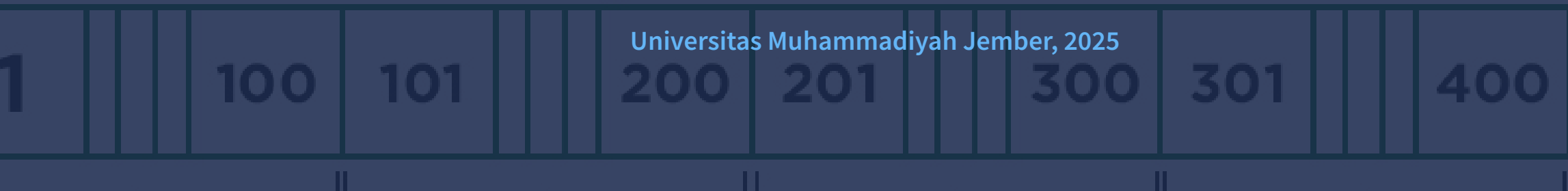
1 sec



Dosen Pengampu: [Triawan Adi Cahyanto, M.Kom](#)

Program Studi Teknik Informatika

with threads QuadCore Machine



t1

t2

t3

t4

X 2
Multiply

Universitas Muhammadiyah Jember, 2025

Daftar Isi

1 Overview



2 Multicore Programming



3 Multithreading Models



4 Thread Libraries



5 Implicit Threading



6 Threading Issues



7 Operating-System Examples



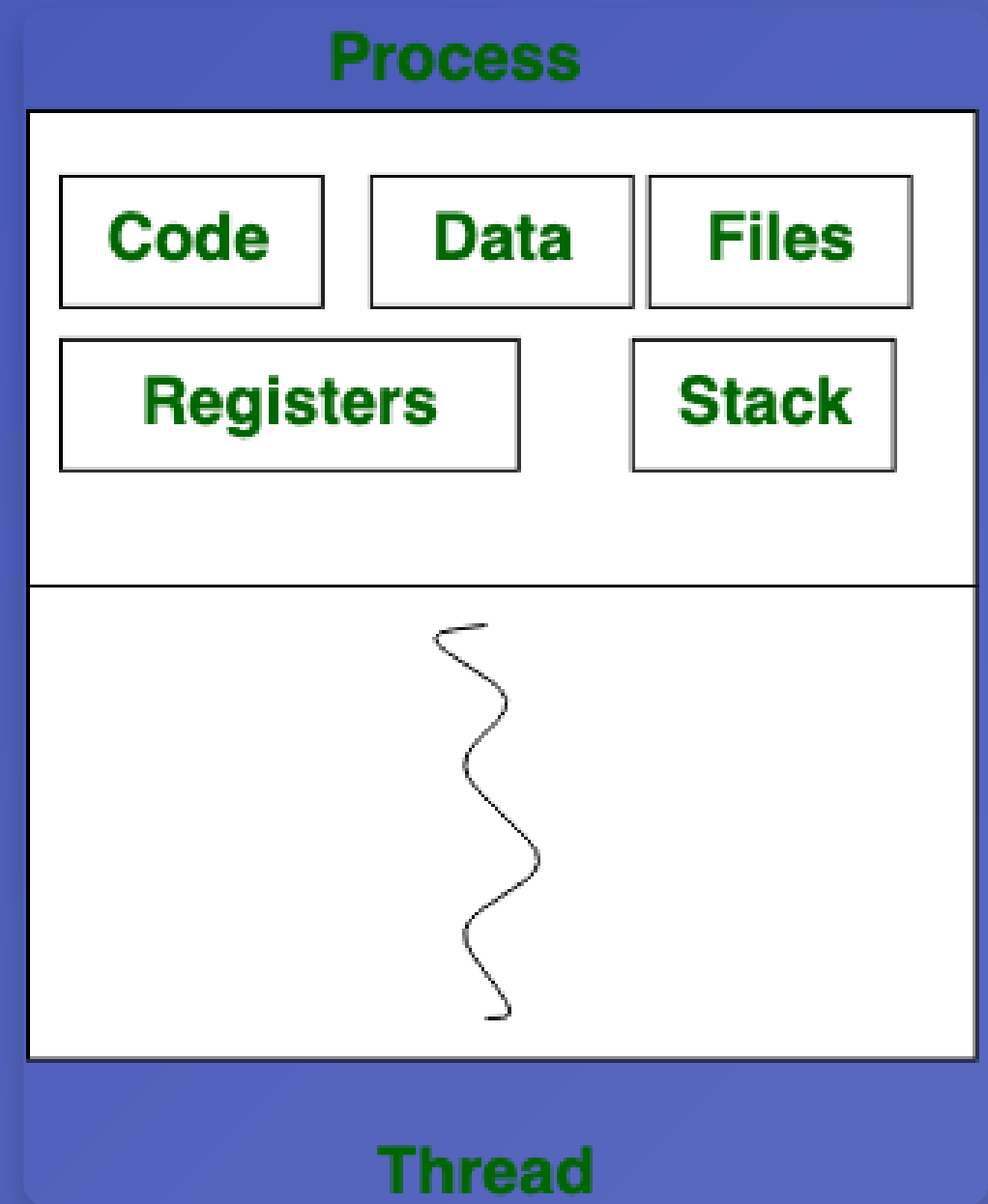
Overview: Konsep Dasar Thread

💡 Apa itu Thread?

Thread adalah unit terkecil dari eksekusi dalam suatu proses yang dapat dijadwalkan oleh sistem operasi.

⚙️ Karakteristik Thread

- Memiliki **program counter**, **register**, dan **stack** sendiri
- Berbagi **kode**, **data**, dan **file** dengan thread lain dalam proses yang sama
- Dapat berjalan **concurrent** atau **parallel**



Overview: Process vs Thread

Process

- ✓ Memiliki ruang alamat sendiri
- ✓ Independen dari proses lain
- ✓ Komunikasi antar proses lebih kompleks
- ✓ Memerlukan lebih banyak resource
- ✓ Konteks switching lebih lambat

Thread

- ✓ Berbagi ruang alamat dalam proses
- ✓ Dependen pada proses induk
- ✓ Komunikasi antar thread lebih mudah
- ✓ Memerlukan lebih sedikit resource
- ✓ Konteks switching lebih cepat

Difference	Process	Thread
Resource Allocation	Allocate new resources each time we run a program.	Share resources of process.
Resource Sharing	In general, resources are not shared. The code may be shared for the same program.	Share code, heap, data area except stack.
Address	Have a separate address space	Share address space
Communication	Communicate through IPC.	Communicate freely with modifying shared variables.
Context Switching	Generally slower than thread.	Generally faster than process.

Overview: Keuntungan Thread

Efisiensi

- Pembuatan thread **lebih cepat** dari proses
- Konteks switching **lebih ringan**
- Penggunaan memori **lebih hemat**

Responsivitas

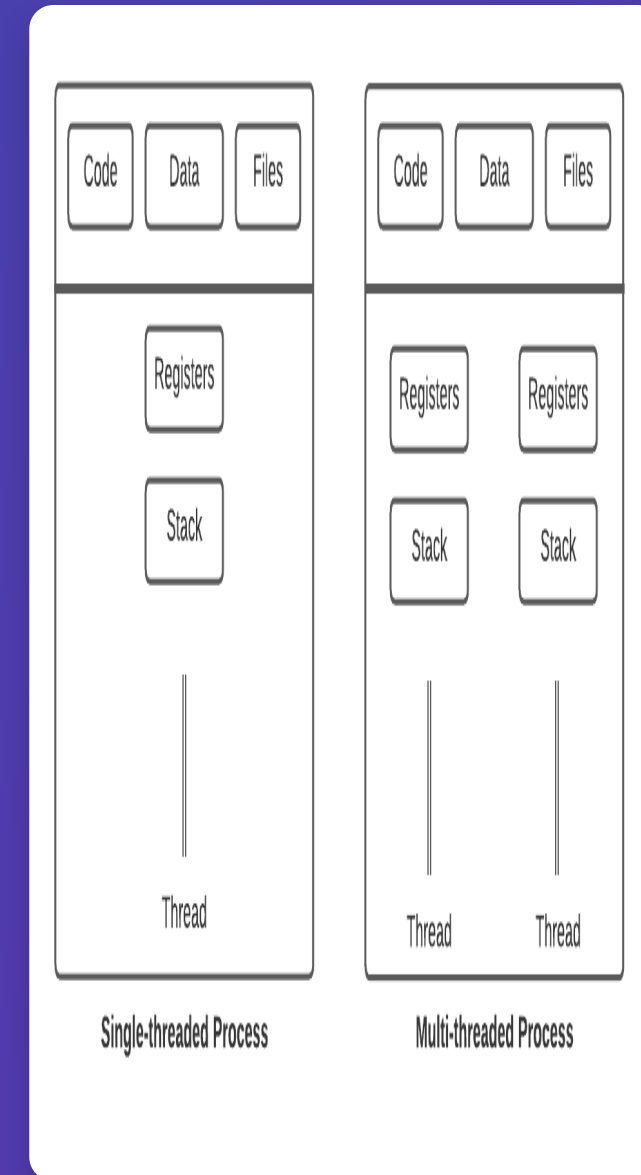
- Aplikasi tetap **responsif** saat ada task berat
- UI tidak **membeku** saat proses berjalan
- Pemrosesan **background** tanpa gangguan

Berbagi Sumber Daya

- Thread dalam proses **berbagi memori**
- Akses file dan resource **lebih mudah**
- Komunikasi antar thread **lebih cepat**

Skalabilitas

- Memanfaatkan **multicore processor**
- Peningkatan performa **linear** dengan core
- Cocok untuk aplikasi **high-performance**



Multicore Programming: Konsep Dasar

🔧 CPU Multicore

Sistem modern dengan CPU multicore memiliki beberapa prosesor independen dalam satu chip, memungkinkan eksekusi **paralel** dari beberapa thread secara bersamaan.

⚡ Keuntungan Multicore

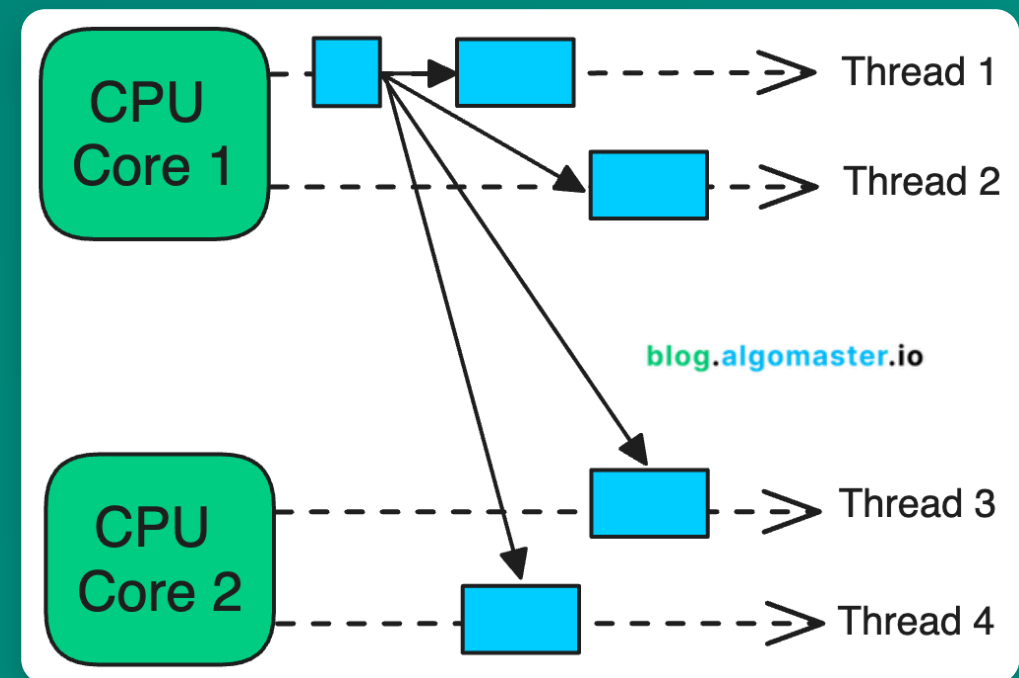
- Performa tinggi untuk aplikasi paralel
- Efisiensi daya lebih baik
- Multitasking yang lebih responsif

⚠️ Tantangan Utama

🏗️ Dividing Tasks

⚖️ Balancing Workload

🔗 Data Sharing



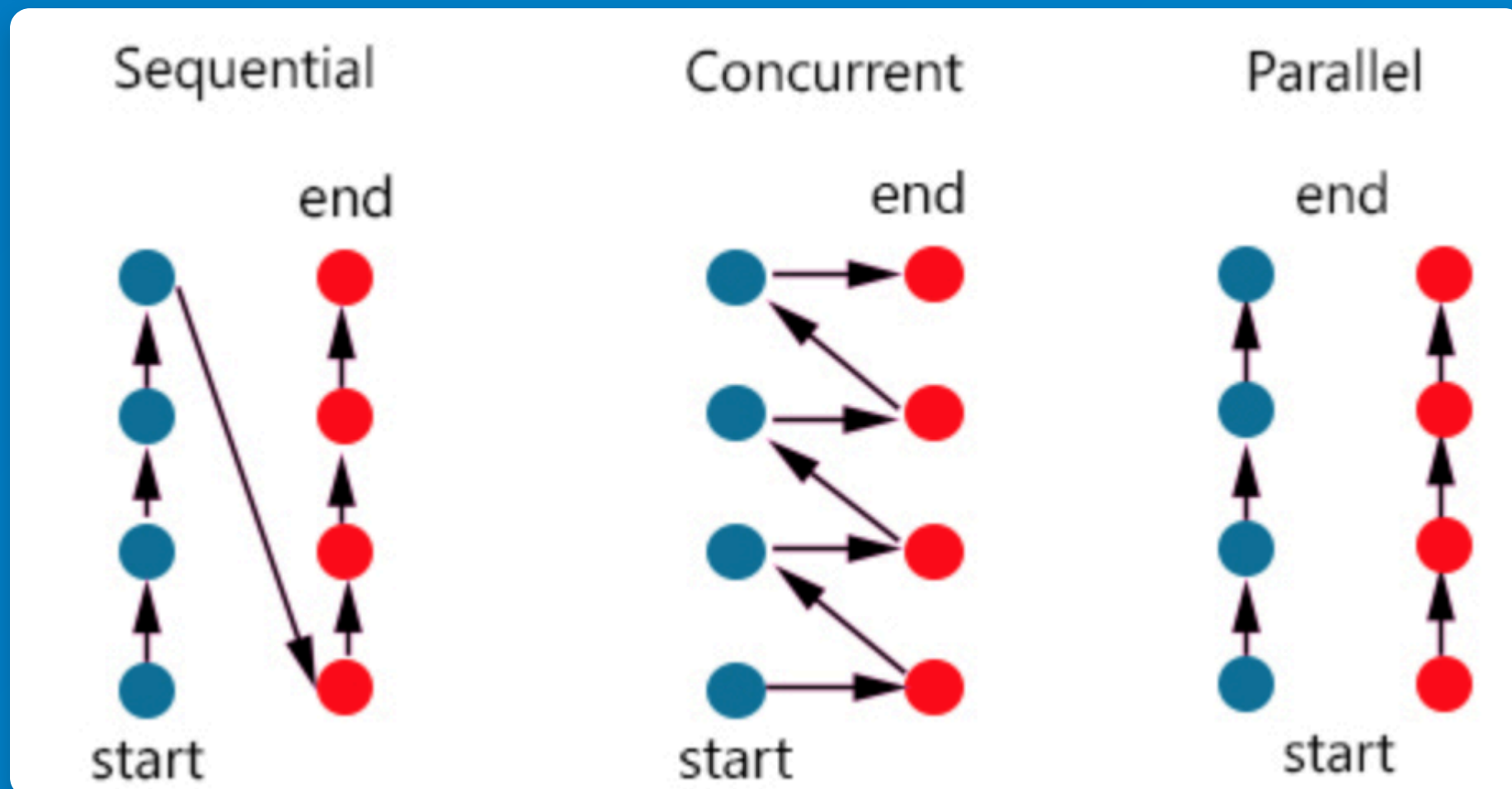
Multicore Programming: Parallelism vs Concurrency

🕒 Concurrency

- Mengatur banyak tugas agar berjalan efisien
- Tugas bergantian dengan cepat
- Bisa terjadi pada single core
- Memberikan ilusi paralelisme

⚡ Parallelism

- Menjalankan banyak tugas sekaligus
- Tugas benar-benar bersamaan
- Memerlukan multicore processor
- Peningkatan performa sejati



Multicore Programming: Model Pemrograman

[] Data Parallelism

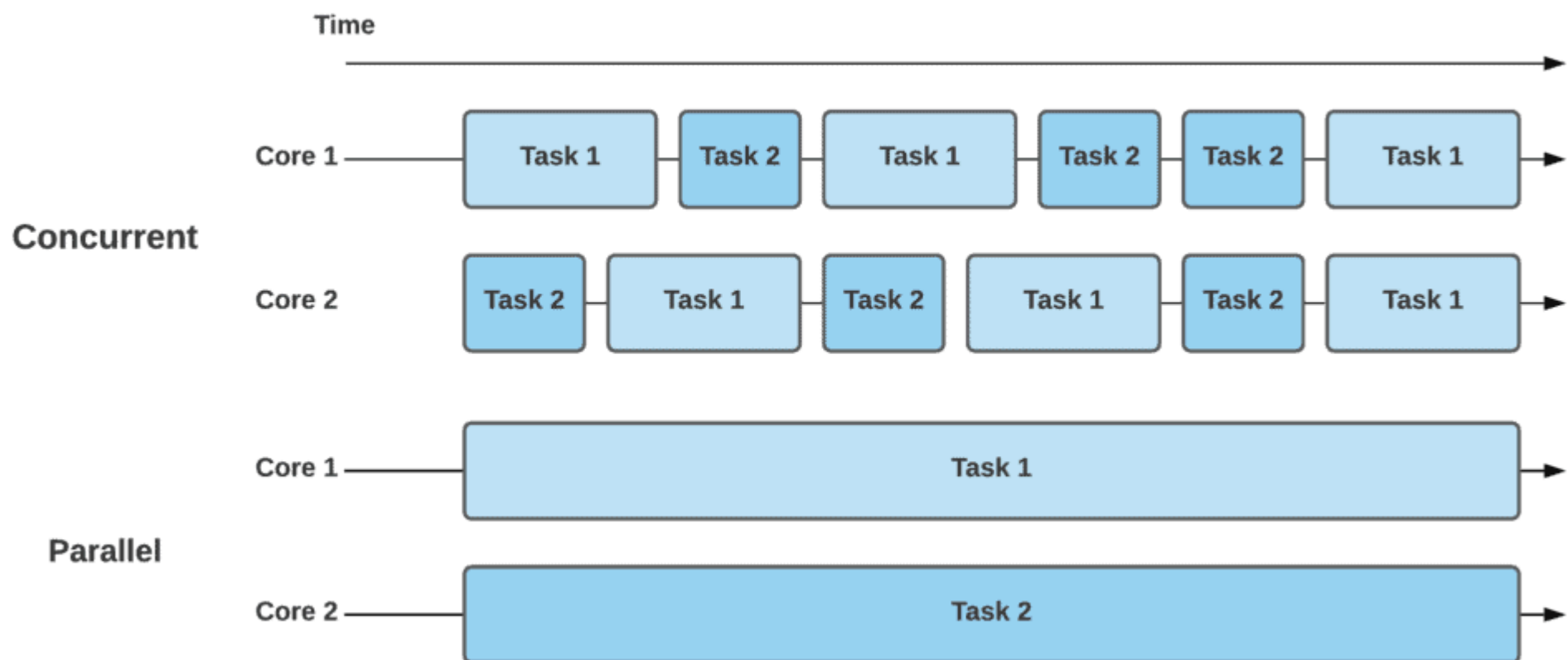
Memproses data yang sama dengan operasi berbeda secara paralel

- ✓ Single instruction, multiple data
- ✓ Setiap core mengerjakan bagian data berbeda
- ✓ Cocok untuk array processing
- ✓ Contoh: GPU computing

✓ Task Parallelism

Menjalankan tugas berbeda secara paralel

- ✓ Multiple instruction, multiple data
- ✓ Setiap core mengerjakan tugas berbeda
- ✓ Cocok untuk pipeline processing
- ✓ Contoh: web server handling requests



Multithreading Models: Pengantar

🗂️ Pemetaan Thread

Sistem operasi memetakan **thread pengguna** ke **thread kernel** untuk eksekusi. Pemetaan ini menentukan bagaimana thread dikelola dan dijadwalkan.

↔️ User Threads vs Kernel Threads

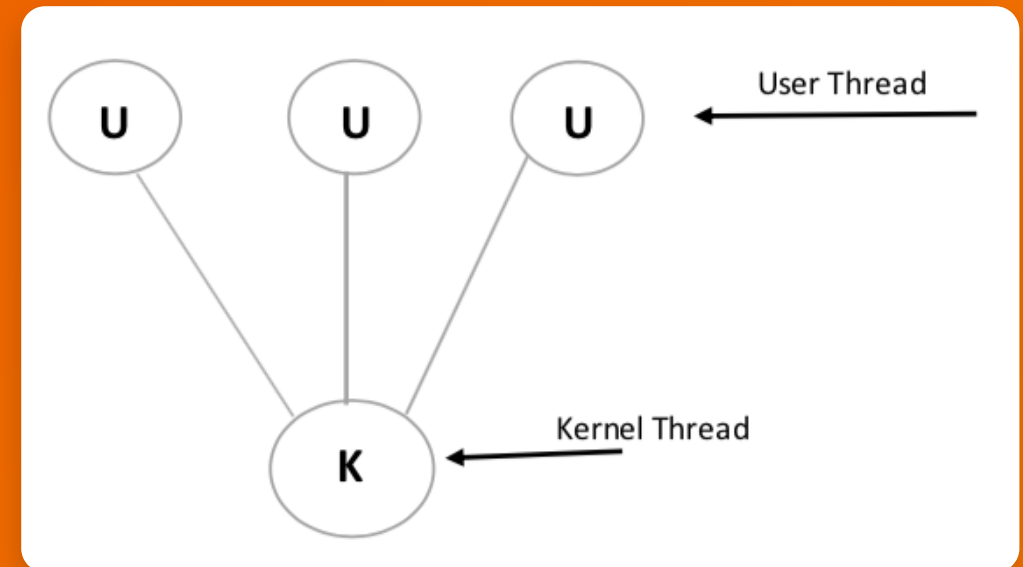
- **User Threads:** Dikelola di ruang pengguna, tidak diketahui oleh OS
- **Kernel Threads:** Dikelola langsung oleh sistem operasi

🏠 Model Threading

1 Many-to-One

2 One-to-One

3 Many-to-Many



Multithreading Models: Many-to-One & One-to-One

1 Many-to-One Model

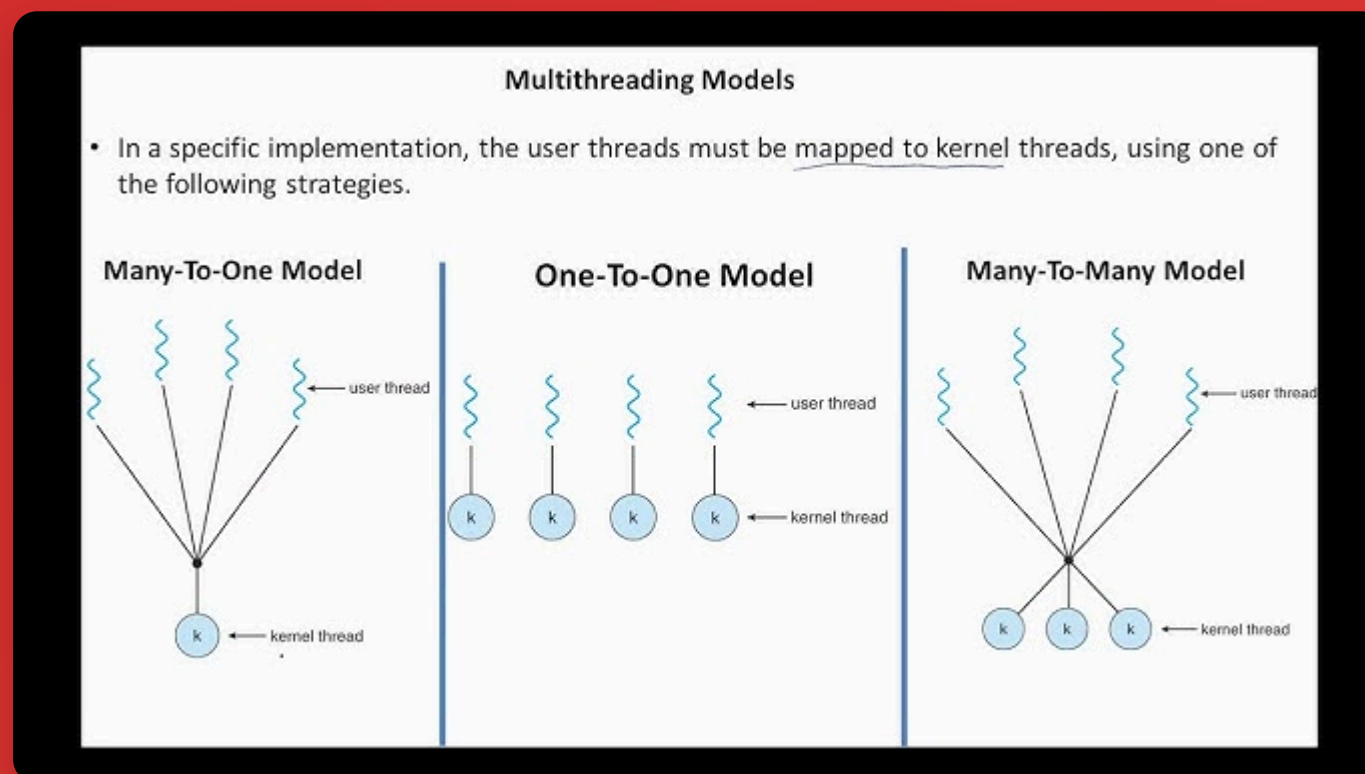
Banyak thread pengguna dijalankan pada satu thread kernel

- ✓ Dikelola di user space
- ✓ Tidak efisien untuk multiprosesor
- ✓ Jika satu thread **blocking**, semua thread terblokir
- ✓ Contoh: Green threads pada Solaris

2 One-to-One Model

Setiap thread pengguna memiliki satu thread kernel

- ✓ Dikelola di kernel space
- ✓ Mendukung multiprosesor
- ✓ Jika satu thread **blocking**, thread lain tetap berjalan
- ✓ Contoh: Windows, Linux, macOS

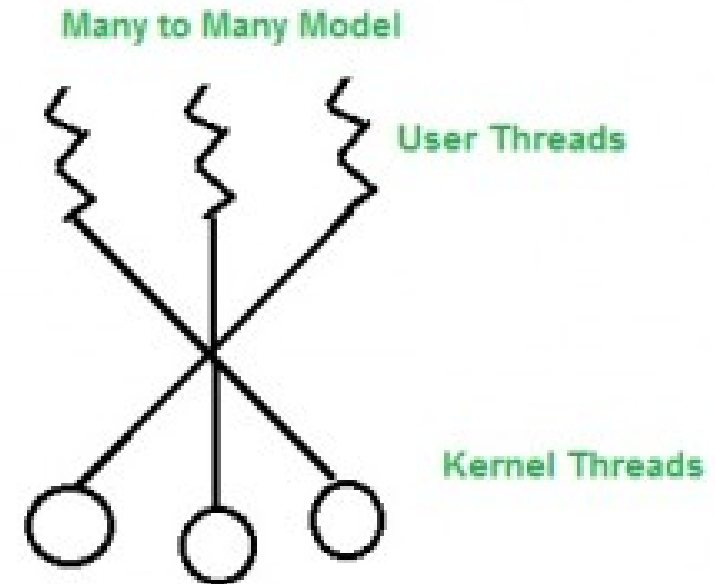


Multithreading Models: Many-to-Many

3 Many-to-Many Model

Kombinasi fleksibel antara Many-to-One dan One-to-One, memungkinkan banyak thread pengguna dipetakan ke banyak thread kernel.

- ✓ Seimbang antara efisiensi dan fleksibilitas
- ✓ Mendukung multiprosesor
- ✓ Thread pengguna **tidak terblokir** jika thread kernel lain blocking
- ✓ Memungkinkan **pengembang** membuat thread sebanyak yang diperlukan
- ✓ Contoh: Solaris, IRIX, Tru64 UNIX



Thread Libraries: POSIX Pthreads & Windows Threads

< > POSIX Pthreads

Pustaka standar untuk sistem **UNIX/Linux** yang menyediakan API untuk pembuatan dan manajemen thread.

- ✓ Standar IEEE untuk thread API
- ✓ Implementasi **kernel-level** threads
- ✓ Fungsi utama: `pthread_create()`, `pthread_join()`

```
pthread_create(&thread, NULL, function, args);
```

⌘ Windows Threads

API thread **native Windows** yang menyediakan fungsi untuk pembuatan dan manajemen thread pada sistem operasi Windows.

- ✓ Menggunakan model **one-to-one**
- ✓ Integrasi dengan **Windows API**
- ✓ Fungsi utama: `CreateThread()`, `WaitForSingleObject()`

```
CreateThread(NULL, 0, function, args, 0, &threadId);
```

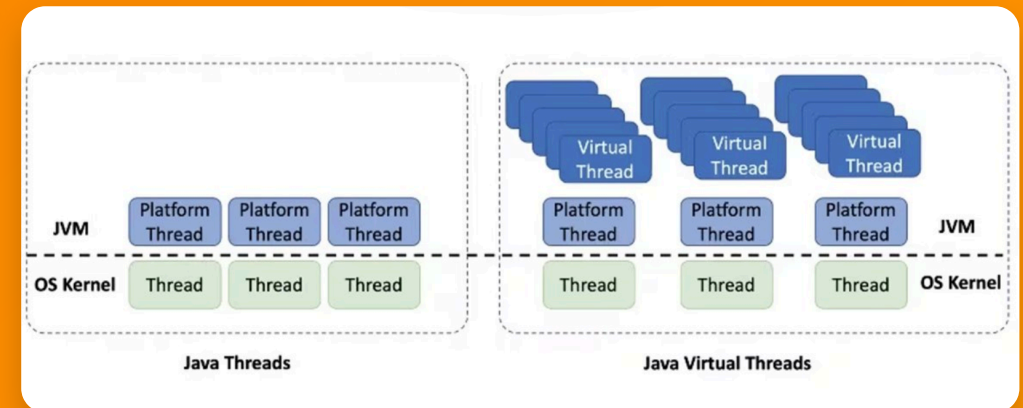
Thread Libraries: Java Threads

☕ Java Threads

Java Threads adalah bagian dari Java Virtual Machine (JVM) yang menyediakan API untuk pembuatan dan manajemen thread dalam aplikasi Java.

- ✓ Platform-independent - kode thread sama di semua OS
- ✓ Menggunakan model **one-to-one** pada JVM modern
- ✓ Dua cara membuat thread: **extend Thread** atau **implement Runnable**

```
Thread thread = new Thread(() -> {  
    // Kode thread  
});  
thread.start();
```



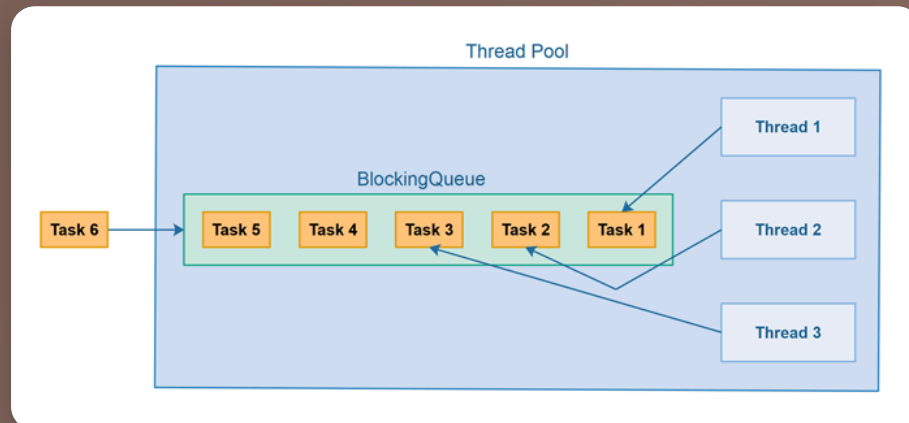
Implicit Threading: Thread Pools & OpenMP



Thread Pools

Thread dibuat di awal dan digunakan ulang untuk mengeksekusi tugas

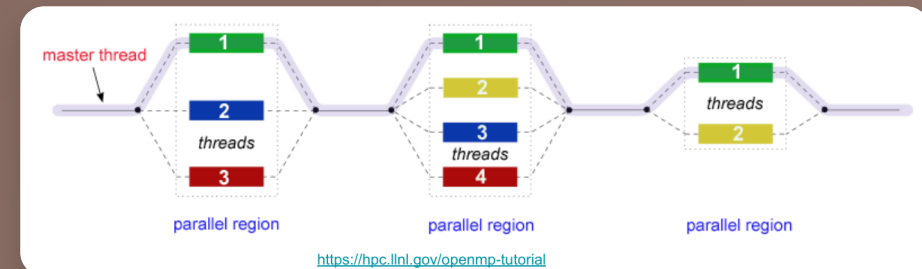
- ✓ Mengurangi overhead pembuatan thread
- ✓ Task queue untuk menyimpan pekerjaan
- ✓ Thread idle menunggu tugas baru
- ✓ Contoh: `ExecutorService` di Java



<> OpenMP

Mendukung paralelisasi otomatis dalam bahasa C/C++/Fortran

- ✓ Compiler directives untuk paralelisasi
- ✓ Fork-join model untuk eksekusi paralel
- ✓ Runtime library mengelola thread
- ✓ Contoh: `#pragma omp parallel for`



Threading Issues: Fork & Exec, Signal Handling

Fork & Exec

Bagaimana thread bereaksi terhadap pembuatan proses baru

- ✓ Fork() - duplikasi proses dengan semua thread
- ✓ Exec() - mengganti program dengan yang baru
- ✓ Masalah - thread di proses anak tidak selalu dibuat
- ✓ Solusi - fork hanya thread yang memanggil

Threading Issues

The fork() and Exec() System Calls

The semantic of the fork() and exec() system calls change in a multithreaded program

Issue

Issue if one thread in a program calls fork(), does the new process duplicate all threads, or is the new process single-threaded?

Solution:

some UNIX system have chosen two versions of fork(), one that duplicates all threads and another that duplicates only the thread that invoked the fork() system call.

But which version of fork() to use and when?

if a thread invokes the exec() system call, the program specified in the parameter to exec() will replace the entire process- including all threads.



Signal Handling

Bagaimana sinyal ditangani oleh thread yang berbeda

- ✓ Sinyal - notifikasi ke proses untuk event tertentu
- ✓ Tantangan - thread mana yang menerima sinyal?
- ✓ Opsi - deliver ke thread tertentu atau semua thread
- ✓ Masking - thread dapat memblokir sinyal tertentu

Threading Issues

Signal Handling

Handling signals in single-threaded programs is straightforward: signals are always delivered to a process. However, delivering signals is more complicated in multithreaded programs, where a process may have several threads. Where, then, should a signal be delivered?

In general, the following options exist:

1. Deliver the signal to the thread to which the signal applies.
2. Deliver the signal to every thread in the process
3. Deliver the signal to certain threads in the process
4. Assign a specific thread to receive all signals for the process

The method for delivering a signal depends on the type of signal generated

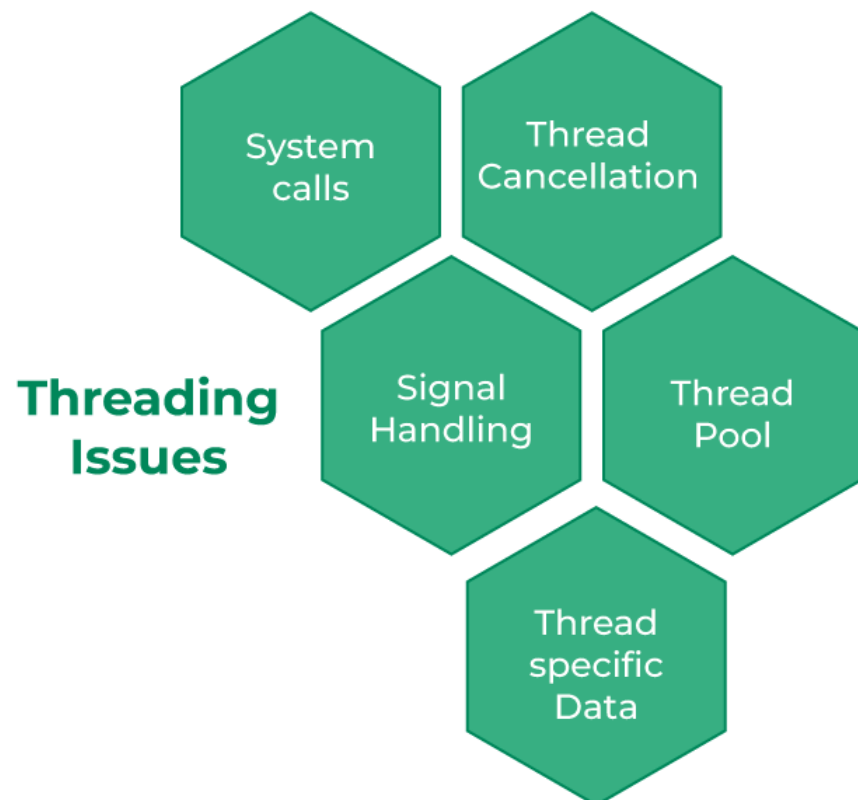
For example, synchronous signals need to be delivered to the thread causing the signal and not to other threads in the process.

Threading Issues: Thread Cancellation & Thread-local Storage

✕ Thread Cancellation

Cara menghentikan thread secara aman sebelum selesai

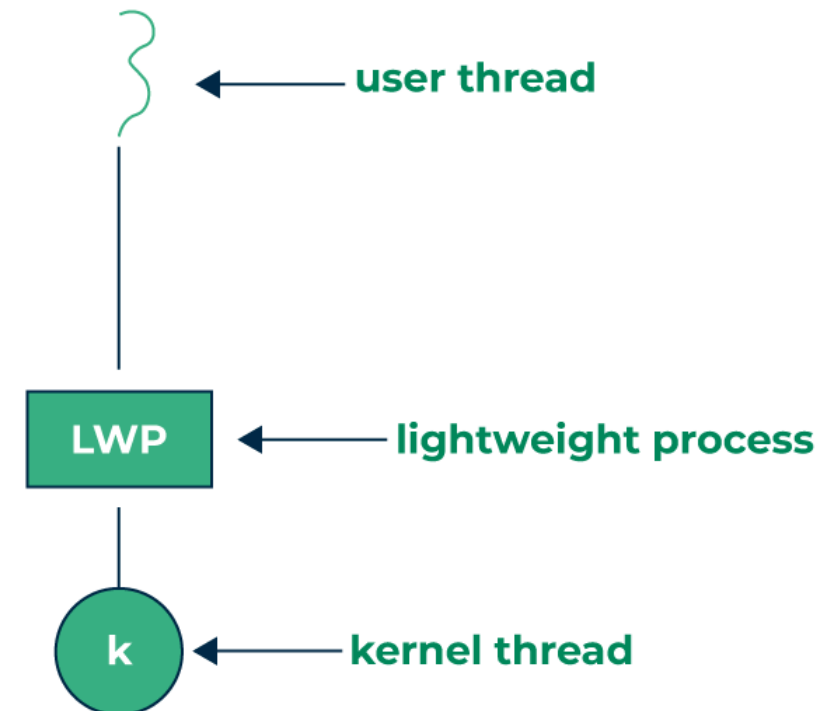
- ✓ Asynchronous - langsung hentikan thread
- ✓ Deferred - thread cek titik pembatalan
- ✓ Masalah - sumber daya mungkin tidak dibersihkan
- ✓ Solusi - gunakan cancellation points



☰ Thread-local Storage

Penyimpanan variabel khusus tiap thread

- ✓ Data privat untuk setiap thread
- ✓ Keuntungan - tidak perlu sinkronisasi
- ✓ Implementasi - TLS (Thread Local Storage)
- ✓ Contoh - errno, thread ID, connection state



Operating-System Examples

Windows

Menggunakan model one-to-one threading dengan API Windows Thread

- ✓ Kernel-level threads
- ✓ Fiber untuk user-level threads
- ✓ Thread Pool API untuk implicit threading

Linux

Menggunakan POSIX Threads (pthreads) dengan kernel-level support

- ✓ Clone() system call untuk pembuatan thread
- ✓ NPTL (Native POSIX Thread Library)
- ✓ Lightweight processes untuk threading

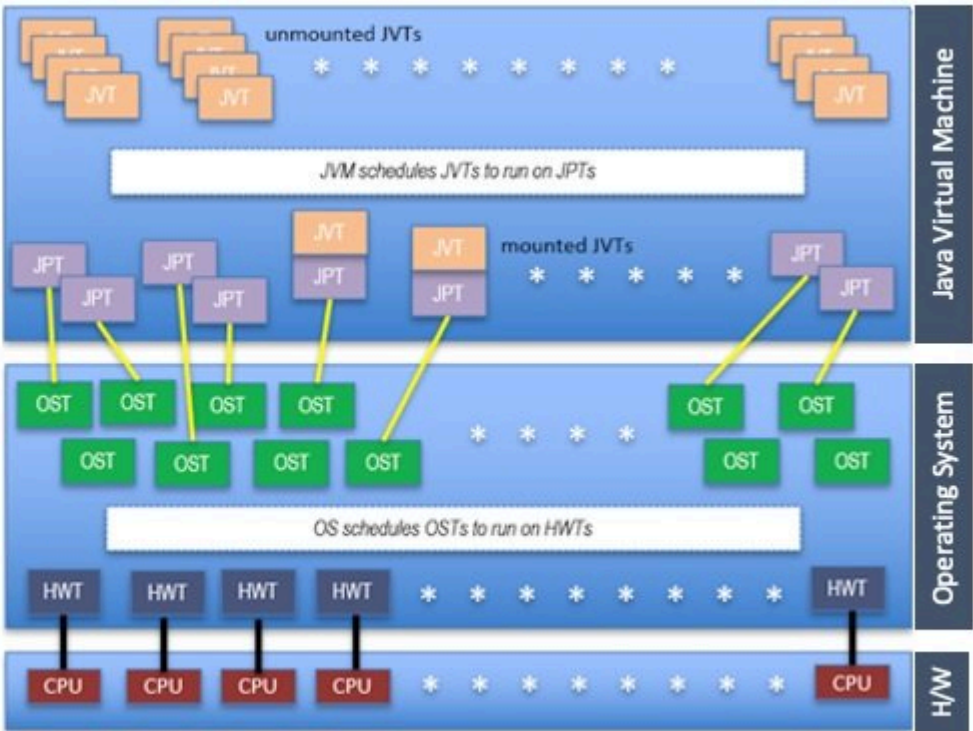
Java

Mengelola thread melalui JVM yang menggunakan native thread di sistem host

- ✓ Platform-independent thread API
- ✓ ExecutorService untuk thread pools
- ✓ Virtual threads pada Java 19+

Java / OS Thread Types

JVT	Java Virtual Thread
JPT	Java Platform Thread
OST	Operating System Thread
HWT	Hardware Thread
CPU	Processor Physical Core



Terima Kasih

